Python Engineering - enumerate() + Error Position Guide

Iterating files, numbering lines, and locating corrupted values in the future NumPy matrix

1. Why do we pass file into enumerate()?

Inside `with open(...) as file`, `file` is an **open file object**. It is not the path; it is the object used to access file contents.

```
with open(file_path, 'r', encoding='utf-8') as file:
    ...
```

A file object is iterable: Python can request the next line repeatedly until end-of-file.

```
for line in file:
    print(line)
```

Therefore it can be the iterable passed to enumerate():

```
for line_number, line in enumerate(file, start=2):
    ...
```

enumerate() needs something iterable. Here, that iterable is the open file itself.

2. file_path and file are different

Variable	Represents	Example
file_path	Where the file is located	C:/project/data/file.csv
file	Open object used to read/write contents	returned by open(...)

enumerate(file_path) would iterate the path representation, not the CSV lines we want to process.

3. What happens after readline()?

```
header = file.readline().strip()
```

This consumes the first line. The file object remembers its current internal position.

```
line 1 -> h_m,v_m_s,t_s      # consumed
line 2 -> 100.0,20.0,0.0      # next
line 3 -> 102.0,BAD,0.1
```

enumerate(file, start=2) therefore labels the first remaining line as physical line 2.

4. start=2 only changes the counter

```
items = ['A', 'B', 'C']
for n, item in enumerate(items, start=2):
    print(n, item)

# 2 A
# 3 B
# 4 C
```

No elements are skipped; only the counter numbering changes.

5. What does enumerate(file, start=2) produce?

```
(2, '100.0,20.0,0.0\n')
(3, '102.0,BAD,0.1\n')
(4, '103.9,18.0,0.2\n')
```

The for loop unpacks each pair into line_number and line.

6. Three different positions - do not mix them

Coordinate	Example	Meaning
Physical CSV line	line_number = 3	Third real file line, including header
Logical data row	row_index = 1	Second data row, zero-based Python index
Array column	column_index = 1	Second column: velocity

For a corrupted value on physical line 3, the future [h,v,t] NumPy location would normally be [1, 1].

7. If float() fails, that cell does not yet exist in the final np.array

```
rows = []
...
rows.append([h, v, t])
...
return np.array(rows)
```

If float('NOT_A_NUMBER') fails, the corrupted row is never appended. Therefore there is not yet an occupied cell in the final NumPy array.

What we can report is the **position it would have occupied** if conversion had succeeded.

8. Simple version when the failing column is known

```
row_index = len(rows)

try:
    v = float(values[1])
except ValueError as error:
    raise ValueError(
        f'Line {line_number}: invalid velocity '
        f'at array position [{row_index}, 1]: '
        f'{values[1]!r}'
    ) from error
```

len(rows) equals the number of valid rows already stored, so it is the next logical row index.

9. Concrete example

```
rows = [[100.0, 20.0, 0.0]]
len(rows) -> 1
line_number -> 3
values[1] -> 'NOT_A_NUMBER'

would-be NumPy position -> [1, 1]

Line 3: invalid velocity at array position [1, 1]: 'NOT_A_NUMBER'
```

10. General version: locate any corrupted column

```
column_names = ['h_m', 'v_m_s', 't_s']
numeric_row = []
row_index = len(rows)

for column_index, value in enumerate(values):
    try:
        numeric_value = float(value)
    except ValueError as error:
        raise ValueError(
            f'Line {line_number}, '
            f'array position [{row_index}, {column_index}], '
            f'column {column_names[column_index]}: '
            f'invalid value {value!r}'
        ) from error

    numeric_row.append(numeric_value)

rows.append(numeric_row)
```

Here enumerate() is used twice for two different numbering systems:

enumerate	What it numbers
enumerate(file, start=2)	Physical lines in the file
enumerate(values)	Columns in the current row, starting at 0

11. What does 'from error' mean?

`raise ... from error` preserves the causal link between the original float() error and our more descriptive engineering error.

```
float('NOT_A_NUMBER') -> original ValueError
raise ValueError('Line 3 ... [1,1] ...') from error
```

It is optional for EX23, but useful when translating an exception into a clearer one.

12. What if blank lines exist?

This is why `row_index = line_number - 2` is not always robust.

```
line 1 -> header
line 2 -> valid row
line 3 -> blank line (skipped)
line 4 -> corrupt row
```

Physical line is 4, but the future NumPy row index is still 1. len(rows) tracks the logical matrix index better.

13. If the array were preallocated

```
trajectory = np.empty((n_rows, 3))
trajectory[row_index, column_index] = value
```

Then the matrix physically exists before all conversions. In our current loader, we report a would-be position because np.array(rows) is created only at the end.

14. CSV line vs NumPy position

```
CSV
line 1 -> header
line 2 -> data row 0 -> NumPy row 0
line 3 -> data row 1 -> NumPy row 1
line 4 -> data row 2 -> NumPy row 2

Columns:
h -> 0
v -> 1
t -> 2
```

Keeping line_number and row_index separate makes the program clearer and more robust.

15. Final cheat sheet

```
file_path -> location
file      -> open iterable file object

enumerate(file, start=2)
-> physical file line + line text

row_index = len(rows)
-> next logical NumPy row

enumerate(values)
-> column index + text value

EXAMPLE
line_number = 3
row_index = 1
column_index = 1
-> future position [1, 1]
```

Core idea: the file line number locates the problem in the source file; [row_index, column_index] expresses where that value would appear in the numerical structure.